UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
INGENIERÍA DE SISTEMAS INFORMÁTICOS

Evaluación

INGENIERÍA DE SOFTWARE

DOCUMENTACIÓN TÉCNICA

Sistema web de pedidos para la cafetería universitaria

Grupo “A”

Estudiante:

PACSI PARI CRISTHIAN GIOVANNI

Docente: Ing. Georgina Aruquipa Gutierrez

La Paz Octubre del 2026

Gestión II – 2026

ÍNDICE DE CONTENIDO

1 INTRODUCCIÓN 1

1.1 Propósito del documento 1

1.2 Estado del proyecto 1

2 HERRAMIENTAS DE DESARROLLO 1

3 ARQUITECTURA DEL SISTEMA 2

3.1 Modelo de datos 2

3.2 Endpoints de la API 3

3.3 Roles y permisos 3

4 REPOSITORIO 4

4.1 Datos generales 4

4.2 Estructura del proyecto 4

5 ESTRATEGIA DE RAMAS 5

5.1 Convención de nombres 5

6 COMMITS 6

6.1 Convención de mensajes 6

6.2 Historial de commits 6

7 PULL REQUESTS 7

7.1 Proceso de revisión 7

8 ISSUES Y TABLERO DE TAREAS 8
8.1 Situación actual 8
8.2 Organización propuesta 8
9 GESTIÓN DE VERSIONES 8
10 INSTALACIÓN Y PUESTA EN MARCHA 9
10.1 Requisitos 9
10.2 Backend (API) 9
10.3 Frontend 10
11 DOCUMENTACIÓN DEL PROYECTO 10
12 MEJORAS RECOMENDADAS AL REPOSITORIO 11
13 CONCLUSIONES 11

Documentación técnica y gestión del repositorio

1 INTRODUCCIÓN

1.1 Propósito del documento

Este documento describe cómo el equipo administra el desarrollo del Sistema web de pedidos para la cafetería universitaria: las herramientas utilizadas, el repositorio y su estructura, la estrategia de ramas, la convención de commits, las solicitudes de integración (pull requests), el seguimiento de tareas, la gestión de versiones y la documentación técnica necesaria para instalar y mantener el sistema. Se complementa con el Manual de usuario, que explica el uso del sistema para cada rol.

1.2 Estado del proyecto

El proyecto se desarrolla con la metodología XP. A la fecha de este documento (21/10/2026) se encuentra cerrando la **iteración 1** (HU-01, HU-02, HU-13, HU-20 y HU-24) y cuenta con avances de iteraciones posteriores (notas en el pedido, número de turno, cambio de estado, cancelación y validación de entrega). El repositorio integra el **backend** (API REST en Laravel) y el **frontend** (HTML, CSS y JavaScript) en una misma rama principal.

2 HERRAMIENTAS DE DESARROLLO

Herramienta	Uso en el proyecto	Categoría
Git	Control de versiones distribuido: ramas, commits y fusiones.	Versionado
GitHub	Repositorio remoto en la organización Cafeteria-Organization, pull requests, revisiones de código e issues.	Repositorio
Visual Studio Code	Editor de código del equipo.	Desarrollo
XAMPP (Apache, MariaDB/MySQL, PHP 8.2)	Servidor local: Apache sirve el frontend; MySQL guarda los datos.	Entorno
Composer	Gestor de dependencias de PHP (Laravel y sus paquetes).	Dependencias
Laravel 9 + Sanctum	Framework del backend y autenticación por token para la API.	Backend
HTML, CSS y JavaScript	Interfaz web sin frameworks, consume la API con fetch.	Frontend
Laravel Pint	Formateo automático del código PHP según PSR-12 (usado en la rama feature/gestion-menu).	Calidad
PHPUnit	Pruebas automatizadas del backend (pruebas de validación de productos).	Pruebas

Herramienta	Uso en el proyecto	Categoría
Postman / navegador	Pruebas manuales de la API y de la interfaz.	Pruebas

Tabla 1. *Herramientas utilizadas por el equipo.*

3 ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura **cliente-servidor desacoplada**: el frontend se ejecuta en el navegador y se comunica con la API mediante peticiones HTTP que intercambian JSON. Las rutas protegidas exigen un token de Laravel Sanctum, que el frontend obtiene al iniciar sesión y envía en la cabecera `Authorization: Bearer`. La API aplica las reglas de negocio con modelos Eloquent y apoya parte de la lógica en procedimientos almacenados y un trigger de MySQL.

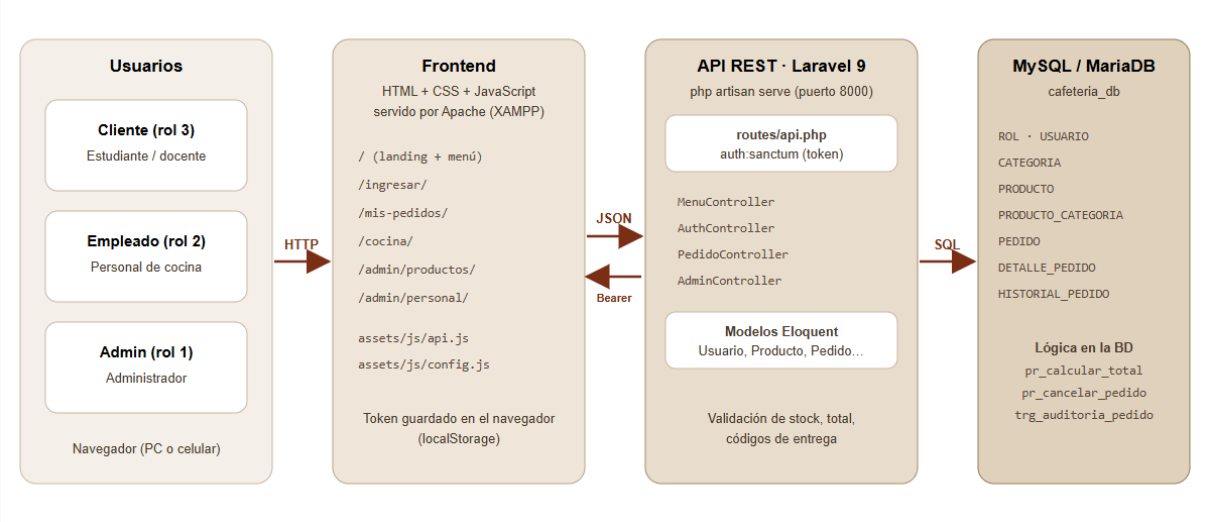


Figura 1. *Arquitectura del sistema y flujo de comunicación.*

3.1 Modelo de datos

Tabla	Contenido
ROL	Roles del sistema: 1 Admin, 2 Empleado, 3 Cliente.
USUARIO	Cuentas: nombre, correo, contraseña cifrada (password_hash), rol y fecha de registro.
CATEGORIA / PRODUCTO	Catálogo: nombre, descripción, precio, stock_actual y estado (Disponible).
PRODUCTO_CATEGORIA	Relación muchos a muchos entre productos y categorías.
PEDIDO	Cabecera del pedido: usuario, fecha, estado, total y código de verificación de 6 caracteres.
DETALLE_PEDIDO	Productos del pedido: cantidad, precio unitario histórico y nota de personalización.

Tabla	Contenido
HISTORIAL_PEDIDO	Auditoría de cambios de estado, llenada por el trigger trg_auditoria_pedido.

Tabla 2. *Tablas principales de la base de datos.*

Lógica en la base de datos: `pr_calcular_total` suma el detalle y actualiza el total del pedido; `pr_cancelar_pedido` solo permite cancelar pedidos en estado Pendiente; `trg_auditoria_pedido` registra cada cambio de estado en HISTORIAL_PEDIDO.

3.2 Endpoints de la API

Método	Ruta	Acceso	Función (HU)
POST	/api/register	Público*	Crear cuenta de cliente; con token de Admin crea personal con rol (HU-24).
POST	/api/login	Público	Iniciar sesión y obtener el token (HU-27).
GET	/api/menu	Público	Productos disponibles con sus categorías (HU-01).
GET	/api/menu/buscar?q=	Público	Búsqueda por nombre (HU-02).
POST	/api/pedidos	Con token	Crear pedido con notas; devuelve número y código (HU-03, HU-04).
GET	/api/pedidos/activos	Con token	Pedidos Pendiente, Preparando y Listo por hora (HU-13).
PATCH	/api/pedidos/{id}/estado	Con token	Cambiar estado o cancelar (HU-14, HU-07).
POST	/api/admin/productos	Con token	Crear producto (HU-20).
PUT	/api/admin/productos/{id}	Con token	Editar producto (HU-20).
GET	/api/admin/personal	Con token	Listar Admin y Empleados (HU-24).
PATCH	/api/admin/usuarios/{id}/rol	Con token	Asignar o quitar rol (HU-24).
GET	/api/mis-notificaciones	Con token	Notificaciones del usuario (HU-06)**.
PATCH	/api/mis-notificaciones/{id}/leer	Con token	Marcar notificación como leída (HU-06)**.

Tabla 3. *Rutas definidas en backend/routes/api.php.*

> *Solo un Admin puede indicar un rol al registrar. **Rutas de notificaciones entregadas por el equipo después de la última integración: deben subirse a `main` mediante un pull request.

3.3 Roles y permisos

Función	Visitante	Cliente	Empleado	Admin
Ver el menú y buscar productos	Sí	Sí	Sí	Sí
Armar el pedido (carrito)	Sí	Sí	No	No
Confirmar pedido y ver “Mis pedidos”	No	Sí	No	No
Ver pedidos de cocina y cambiar su estado	No	No	Sí	Sí
Crear y editar productos	No	No	No	Sí
Crear cuentas de personal y asignar roles	No	No	No	Sí

Tabla 4. *Matriz de permisos por rol.*

> Hoy los permisos se aplican en la interfaz. En la API las rutas de administración y de cocina solo exigen sesión, no rol (defecto DEF-01 del plan de pruebas). Se recomienda un middleware de rol en Laravel.

4 REPOSITORIO

4.1 Datos generales

Dato	Valor
URL	https://github.com/Cafeteria-Organization/cafeteria-pedidos
Organización	Cafeteria-Organization
Visibilidad	Público
Rama principal	main
Creación	24/09/2026
Última integración	20/10/2026 (PR #4)
Contenido	14 commits, 6 ramas, 4 pull requests fusionados

Tabla 5. *Datos del repositorio (consultados el 21/10/2026).*

4.2 Estructura del proyecto

Desde el PR #4 el repositorio separa claramente las dos partes del sistema y reserva una carpeta para la documentación:

```
cafeteria-pedidos/
├── backend/                 API REST en Laravel 9
│   ├── app/Http/Controllers/Api/   AuthController, MenuController,
│   │                               PedidoController, AdminController
│   ├── app/Models/          Usuario, Rol, Producto, Categoria, Pedido...
│   ├── database/migrations/ Tablas, procedimientos y trigger
│   ├── database/seeders/    Roles, administrador y datos iniciales
│   ├── routes/api.php       Endpoints de la API
│   └── tests/               Pruebas automatizadas (PHPUnit)
├── frontend/                Interfaz web (HTML, CSS, JS)
│   ├── index.html           Landing con el menú
│   ├── ingresar/  mis-pedidos/  cocina/
│   ├── admin/productos/  admin/personal/
│   ├── assets/css/  assets/js/   Estilos y lógica (config.js, api.js...)
│   └── version-anterior-php/     Primera versión (PHP sin API)
├── docs/                    Documentación del proyecto
├── .gitignore
└── README.md
```

Figura 2. *Estructura de carpetas de la rama main.*

5 ESTRATEGIA DE RAMAS

El equipo trabaja con un flujo basado en **ramas de funcionalidad**: cada historia o cambio se desarrolla en su propia rama a partir de `main` y se integra mediante un pull request. La rama `develop` se creó como rama de integración en la primera semana.

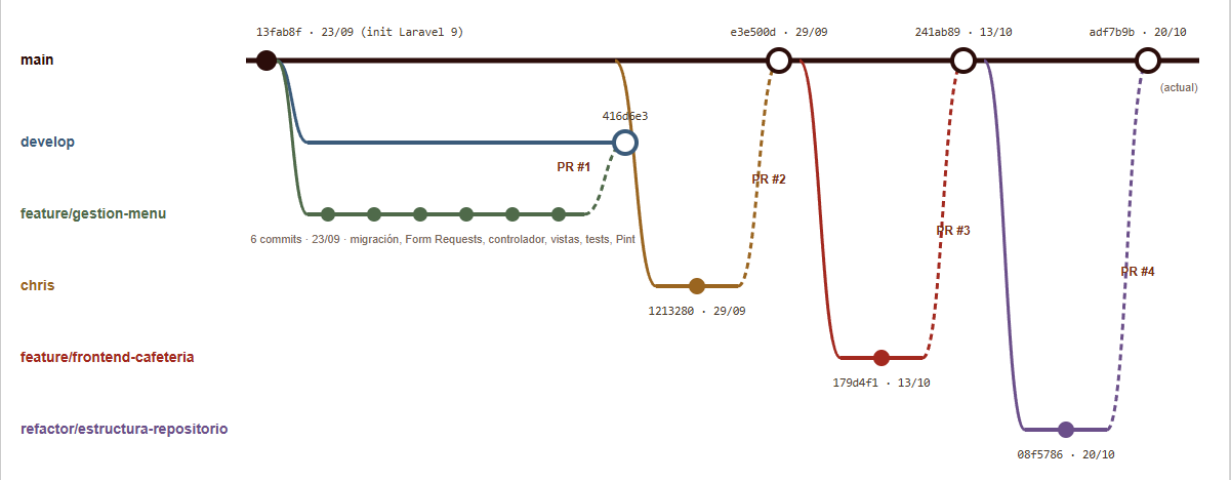


Figura 3. *Historial de ramas y fusiones del repositorio (todas las ramas).*

Rama	Propósito	Estado
main	Versión estable e integrada del sistema.	Activa (rama principal)
develop	Integración de funcionalidades antes de pasar a main.	Recibió el PR #1; no se fusionó a main
feature/gestion-menu	CRUD de productos con vistas Blade y pruebas (HU-20).	Fusionada a develop (PR #1)
chris	Integración de la primera iteración (HU-01, 02, 13, 20, 24, 26, 27).	Fusionada a main (PR #2)
feature/frontend-cafeteria	Frontend que consume la API.	Fusionada a main (PR #3)
refactor/estructura-repositorio	Reorganización en backend/, frontend/ y docs/.	Fusionada a main (PR #4)

Tabla 6. *Ramas del repositorio.*

5.1 Convención de nombres

- `feature/<descripcion>`: nueva funcionalidad o historia de usuario (ej.: feature/gestion-menu).
- `fix/<descripcion>`: corrección de un defecto (ej.: fix/control-rol-api).
- `refactor/<descripcion>`: cambios de estructura sin cambiar el comportamiento.
- `docs/<descripcion>`: documentación.

- Se recomienda no usar ramas con nombre de persona (como `chris`) y renombrarlas según su contenido.

6 COMMITS

6.1 Convención de mensajes

Los mensajes siguen la convención **Conventional Commits**: `tipo(alcance): descripción`, en español, en modo imperativo y, cuando corresponde, con la historia de usuario entre paréntesis.

Tipo	Uso	Ejemplo real del repositorio
feat	Nueva funcionalidad	feat(menu): implementar ProductoController y rutas (HU-20)
test	Pruebas automatizadas	test(menu): validar campos obligatorios y precio no negativo (HU-20)
style	Formato sin cambiar la lógica	style(menu): aplicar Laravel Pint (PSR-12)
refactor	Reestructuración del código	refactor: reorganizar repositorio con estructura sugerida
chore	Configuración y tareas de mantenimiento	chore: inicializar proyecto Laravel 9
merge	Integración de ramas	merge(integracion): unificar versión… (HU-01, HU-02, HU-13…)
fix / docs	Corrección de errores / documentación	(a usar en las siguientes iteraciones)

Tabla 7. *Tipos de commit utilizados.*

6.2 Historial de commits

Commit	Fecha	Autor (GitHub)	Mensaje
13fab8f	23/09	sassan225	chore: inicializar proyecto Laravel 9
f37fb4b	23/09	sassan225	feat(menu): agregar migración y modelo Producto (HU-20)
6038a12	23/09	sassan225	feat(menu): agregar Form Requests de producto (HU-20)
dca9818	23/09	sassan225	feat(menu): implementar ProductoController y rutas (HU-20)
a8728d8	23/09	sassan225	feat(menu): crear vistas productos.index y productos.create (HU-20)
5fa06fa	23/09	sassan225	test(menu): validar campos obligatorios y precio no negativo (HU-20)
8d8772b	23/09	sassan225	style(menu): aplicar Laravel Pint (PSR-12)
416d6e3	23/09	Chrissho	Merge pull request #1 (feature/gestion-menu → develop)
1213280	29/09	chrischidori27-debug	merge(integracion): unificar versión con la primera iteración
e3e500d	29/09	Chrissho	Merge pull request #2 (chris → main)
179d4f1	13/10	Cristhian	feat(frontend): agregar frontend HTML/CSS/JS que consume la API
241ab89	13/10	Lukas Llanos Medrano	Merge pull request #3 (feature/frontend-cafeteria → main)
08f5786	20/10	sassan225	refactor: reorganizar repositorio (backend/frontend/docs)
adf7b9b	20/10	Lukas Llanos Medrano	Merge pull request #4 (refactor/estructura-repositorio → main)

Tabla 8. *Los 14 commits de todas las ramas (git log --all).*

Cuenta(s) de Git	Integrante (por correo)	Commits
sassan225 / Lukas Llanos Medrano	Lukas Llanos Medrano	10
Chrissho / chrischidori27-debug	Chris Jeremy Rojas Condori	3
Cristhian (cristhianvk-bit)	Cristhian Giovanni Pacsi Pari	1

Tabla 9. *Commits por integrante (incluye commits de fusión).*

> Para que cada aporte quede registrado, todos los integrantes deben subir su trabajo con su propia cuenta y configurar `git config user.name` y `user.email` con su cuenta de GitHub.

7 PULL REQUESTS

Ningún cambio se sube directamente a `main`: todo se integra mediante un pull request (PR) desde una rama, lo que deja registro de qué se cambió, quién lo propuso y quién lo revisó.

PR	Título	Ramas	Autor	Revisión / fusión
#1	feat(menu): CRUD de productos (HU-20)	feature/gestion-menu → develop	sassan225	Comentario de chrischidori27-debug; fusionado 24/09
#2	merge(integracion): unificar la primera iteración	chris → main	chrischidori27-debug	Fusionado 29/09
#3	feat(frontend): agregar frontend HTML/CSS/JS que consume la API	feature/frontend-cafeteria → main	cristhianvk-bit	Revisores: Jammy-QRA, sassan225, chrischidori27-debug; fusionado 13/10
#4	refactor: reorganizar repositorio con estructura sugerida	refactor/estructura-repositorio → main	sassan225	Fusionado 20/10

Tabla 10. *Pull requests del repositorio (todos fusionados).*

7.1 Proceso de revisión

1. Crear la rama desde `main` actualizada: `git switch -c feature/<nombre>`.
2. Hacer commits pequeños con la convención del punto 6.
3. Subir la rama (`git push -u origin feature/<nombre>`) y abrir el PR hacia `main` con una descripción de qué historia cubre y cómo probarlo.
4. Asignar al menos un revisor del equipo; el revisor prueba el cambio y aprueba o pide correcciones.
5. Fusionar solo con la aprobación y borrar la rama al terminar.

> Mejora recomendada: activar en GitHub la protección de `main` (Settings → Branches) para exigir al menos una aprobación antes de fusionar. Los PR #2 y #4 se fusionaron alrededor de un minuto después de abrirse y sin revisión registrada; el PR #3 tenía tres revisores asignados, pero no registra aprobaciones.

8 ISSUES Y TABLERO DE TAREAS

8.1 Situación actual

Al 21/10/2026 el repositorio **no registra issues** y no se encontró un tablero de GitHub Projects vinculado (consulta a la API pública de GitHub; un tablero privado de la organización no sería visible). El seguimiento de las historias se lleva en el documento del proyecto (backlog e iteraciones).

8.2 Organización propuesta

Para que el avance sea visible y trazable desde GitHub se propone:

- **Un issue por historia de usuario**, con su descripción "Como… quiero… para…", criterios de aceptación como lista de verificación y la iteración asignada.
- **Un issue por defecto** encontrado en las pruebas (DEF-01 a DEF-09 del plan de pruebas), con la etiqueta `bug` y su severidad.
- **Etiquetas:** `historia`, `bug`, `backend`, `frontend`, `prioridad-alta/media/baja`.
- **Milestones:** Iteración 1 a Iteración 5, para ver el avance de cada iteración.
- **Tablero (GitHub Projects)** con columnas Backlog → Por hacer → En progreso → En revisión → Hecho; cada PR se enlaza a su issue escribiendo `Closes #número` en la descripción para cerrarlo automáticamente al fusionar.

Issue	Título	Milestone	Columna inicial
HU-01	Consulta del menú digital	Iteración 1	Hecho
HU-02	Búsqueda de productos por nombre	Iteración 1	Hecho
HU-13	Recepción organizada de pedidos	Iteración 1	Hecho
HU-20	Creación y edición de productos	Iteración 1	En progreso (falta eliminar)
HU-24	Gestión de roles y permisos	Iteración 1	En progreso (falta control de rol en API)
HU-03/04/14	Notas, número de turno y cambio de estado	Iteración 2	En revisión
HU-06	Notificación de pedido listo	Iteración 2	En progreso
DEF-01	Las rutas de administración no verifican el rol	Iteración 1	Por hacer (crítico)

Tabla 11. *Ejemplo de issues iniciales para el tablero.*

9 GESTIÓN DE VERSIONES

Se adopta el **versionado semántico** `MAYOR.MENOR.PARCHE`: MAYOR para cambios incompatibles, MENOR para nuevas funcionalidades e iteraciones, y PARCHE para

correcciones. Cada versión se marca con una **etiqueta (tag)** en `main` y se publica como **Release** en GitHub con sus notas de cambios.

El repositorio todavía no tiene etiquetas ni releases. A partir de su historial se propone marcar las versiones así:

Versión	Commit	Fecha	Contenido
v0.1.0	13fab8f	23/09	Proyecto Laravel 9 inicial.
v0.2.0	e3e500d	29/09	Backend de la iteración 1: menú, búsqueda, pedidos, cocina, productos y roles.
v0.3.0	241ab89	13/10	Frontend web que consume la API.
v0.3.1	adf7b9b	20/10	Reorganización del repositorio (backend/, frontend/, docs/).
v1.0.0	(próximo)	—	Cierre de la iteración 1 con los defectos críticos y altos corregidos.

Tabla 12. *Versiones propuestas a partir del historial.*

Comandos para crear y publicar una etiqueta (por ejemplo, la versión actual):

```
git switch main
git pull
git tag -a v0.3.1 adf7b9b -m "v0.3.1: reorganización del repositorio"
git push origin v0.3.1
```

Luego, en GitHub → Releases → *Draft a new release*, se elige la etiqueta y se escriben las notas de la versión. Se recomienda mantener además un archivo `CHANGELOG.md` en la raíz con el resumen de cambios de cada versión.

10 INSTALACIÓN Y PUESTA EN MARCHA

10.1 Requisitos

- XAMPP con Apache, MySQL/MariaDB y PHP 8.0.2 o superior (el equipo usa PHP 8.2).
- Composer y Git.
- Navegador actualizado (Chrome o Edge).

10.2 Backend (API)

```
git clone https://github.com/Cafeteria-Organization/cafeteria-pedidos.git
cd cafeteria-pedidos/backend
composer install
copy .env.example .env
php artisan key:generate
```

En `.env` se configura la base de datos (`DB_DATABASE=cafeteria_db`, `DB_USERNAME=root`). Después se crea la base de datos vacía en phpMyAdmin y se ejecuta:

```
php artisan migrate --seed
php artisan serve
```

La API queda disponible en `http://127.0.0.1:8000/api`. El seeder crea los roles y la cuenta de administrador inicial.

> Para actualizar una base de datos que ya tiene información se usa `php artisan migrate` (aplica solo las migraciones nuevas). `migrate:fresh` borra todas las tablas y los datos.

10.3 Frontend

1. Copiar la carpeta `frontend/` dentro de `C:\xampp\htdocs\` con el nombre deseado (por ejemplo `CAFETERIA_5`).
2. Si la API no está en `http://127.0.0.1:8000/api`, cambiar `API_URL` en `assets/js/config.js`.
3. Con Apache encendido, abrir `http://localhost/CAFETERIA_5/`.

> Al publicar cambios del frontend se incrementa el número de versión de los archivos (`app.css?v=...`) en las páginas HTML; así los navegadores cargan la versión nueva y no una copia en caché.

11 DOCUMENTACIÓN DEL PROYECTO

Documento	Contenido
Informe del proyecto	Problema, justificación, objetivos, historias de usuario, backlog e iteraciones (XP).
Plan de pruebas	Estrategia, criterios de calidad y matriz de 38 casos de prueba con resultados.
Documentación técnica (este documento)	Herramientas, arquitectura, API, repositorio, ramas, commits, PR, issues y versiones.
Manual de usuario	Uso del sistema paso a paso para cliente, empleado y administrador.
frontend/README.md	Páginas, historias cubiertas y estructura del frontend.

Tabla 13. *Documentación disponible.*

Se recomienda guardar estos documentos en la carpeta `docs/` del repositorio y reemplazar el `README.md` de la raíz (que aún es el texto por defecto de Laravel) por una descripción del proyecto con los pasos de instalación.

12 MEJORAS RECOMENDADAS AL REPOSITORIO

N°	Situación encontrada	Acción
1	El `.gitignore` ignora `/vendor` en la raíz, pero tras el PR #4 las dependencias quedaron en `backend/vendor` y se versionaron 7 393 archivos.	Agregar `backend/vendor/` al .gitignore y ejecutar `git rm -r --cached backend/vendor`.
2	No hay issues ni tablero de tareas.	Crear issues por historia y defecto, milestones por iteración y un tablero en GitHub Projects.
3	No hay etiquetas ni releases.	Etiquetar v0.1.0 a v0.3.1 y publicar releases con notas.
4	`main` no exige revisión antes de fusionar.	Activar la protección de rama con al menos una aprobación.
5	La rama `develop` quedó desactualizada y la rama `chris` tiene nombre de persona.	Decidir si se usa develop como rama de integración o se elimina; renombrar o borrar ramas ya fusionadas.
6	README de la raíz por defecto y carpeta `docs/` vacía.	Escribir el README del proyecto y subir la documentación a docs/.
7	Las rutas de notificaciones aún no están en main.	Integrarlas con un pull request.

Tabla 14. *Mejoras sugeridas para la siguiente iteración.*

13 CONCLUSIONES

- El equipo cuenta con un repositorio organizado en backend, frontend y documentación, y todas las integraciones a la rama principal se realizaron mediante pull requests.
- Los commits siguen una convención clara (Conventional Commits) que permite relacionar cada cambio con su historia de usuario.
- Quedan pendientes prácticas de gestión que GitHub ofrece y que harán visible el avance: issues, tablero de tareas, etiquetas de versión, protección de la rama principal y la corrección del .gitignore.
- La documentación técnica, el plan de pruebas y el manual de usuario permiten instalar, verificar y usar el sistema sin depender de una persona en particular.